



Санкт-Петербургский государственный университет
Кафедра системного программирования

Динамическая бинарная трансляция в RISC-V с помощью Instrew

Михайлов Илья Игоревич, группа 21.Б07-мм

Научный руководитель: к.ф.-м.н. Д.В. Луцив, доцент кафедры системного программирования

Консультант: В.А. Кутуев, инженер-исследователь, Лаборатория технологий
программирования инфраструктурных решений СПбГУ

Санкт-Петербург
2025

- Бинарные трансляторы являются инструментами для анализа кода, профилиции и эмуляции
- Важным классом которых являются динамические бинарные трансляторы (далее — ДБТ)
- В качестве промежуточного представления для ДБТ хорошо использовать LLVM IR
 - ▶ Тулчейн LLVM позволяет проводить качественные оптимизации
- Хотим ДБТ для RISC-V

- Динамические бинарные трансляторы:
 - ▶ QEMU-user
 - ▶ Rosetta 2
 - ▶ Valgrind
 - ▶ Box64
- Динамические бинарные трансляторы с поднятием в LLVM IR:
 - ▶ Instrew + Rellume
 - ▶ BinRec, HQEMU (не обновлялись с 2022 и 2018 года соответственно)
 - ▶ DBILL, LnQ, Risotto (есть только статьи, либо код закрыт)

- Выводы:

- ▶ Instrew производительнее, чем QEMU и Valgrind (основываясь на бенчмарках¹)
- ▶ Instrew "из коробки" имеет API для добавления инструментов бинарного анализа
- ▶ Из всех рассмотренных ДБТ с поднятием в LLVM IR Instrew + Rellume поддерживается мейнтейнером
- ▶ Instrew уже имеет guest-поддержку RISC-V

¹<https://mediatum.ub.tum.de/doc/1614897/1614897.pdf>

Постановка задачи

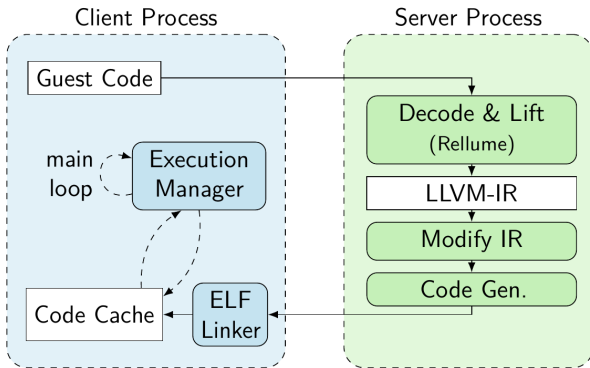
Цель: Добавить поддержку host-архитектуры RISC-V для ДБТ Instrew

Задачи:

- 1 Протестировать на произвольных статически слинкованных программах
- 2 Исправить ошибки, выявленные при тестировании
- 3 Добавить в CI поддержку сборки на RISC-V

Архитектура Instrew

- Является клиент-серверной



2

² диаграмма была взята [отсюда](#)

Что было сделано

Было сделано в предыдущей работе:

- Выполнен обзор архитектуры Instrew
- Реализованы процессорно-специфические патчи
- Реализована эмуляция системных вызовов, PLT трамплины и минимальный набор ELF релокаций
- Instrew был протестирован на статически слинкованных программах, с простым рантаймом

Тестирование (1/2)

- Для выявления ошибок и улучшения поддержки RISC-V-хоста
- Используем следующий подход:
 - ▶ Выбираем набор тестовых программ
 - ▶ Компилируем статически и динамически
 - ▶ В том числе компилируем под разные платформы
 - ▶ Выявляем ошибки и исправляем их
- В процессе тестирования было сделано:
 - ▶ Добавлены новые релокации и обновлена PLT-таблица
 - ▶ Сделан рефакторинг
- Из трудностей можно выделить:
 - ▶ Частично тестирование проводилось на плате Sipeed Lichee PI 4A, где был установлен специфический glibc, в следствие чего было потрачено время на поиск плохо задокументированных инструкций, которые Rellume не мог поднять в LLVM IR

- Instrew использует Rellume в качестве git submodule
- При запуске Instrew unit-тестов также запускаются unit-тесты Rellume
 - ▶ Некоторые тест-кейсы используют LLVM JIT, которые падают с SEGFAULT на RISC-V
- Хотим попробовать обновить версию LLVM до 21, что потенциально может решить проблему
 - ▶ Кросс-компилируем LLVM trunk под RISC-V
 - ▶ Делаем рефакторинг

- Есть трудности в связи с отсутствием нативной поддержки RISC-V в GH Actions
- Есть два варианта решения этой проблемы:
 - ▶ С помощью self-hosted runner на RISC-V
 - ▶ С помощью qemu-user и chroot
- Удалось опробовать оба решения, но был выбран второй подход, так как:
 - ▶ Есть сложности с поддержкой self-hosted runner на RISC-V
 - ▶ Удалось найти third-party action, с помощью которого не нужно настраивать qemu-user и chroot
- Также после добавления CI был сделан PR в основную ветку Instrew

В результате работы были выполнены следующие задачи:

- 1 Instrew был протестирован на произвольных статически слинкованных программах
- 2 Исправлены некоторые ошибки, выявленные при тестировании
- 3 В CI добавлена поддержка сборки на RISC-V

Код доступен по данной ссылке:

<https://github.com/aengelke/instrew/pull/20>.